Use Phantocite in eduGenAI 2

Give an eduGenAI 2 agent an Action that verifies a paper's references against OpenAlex — inside your own chat, with no API key of your own.

eduGenAI 1 was withdrawn after vulnerabilities were found in an audit, and its Extension builder went with it. eduGenAI 2 (edugenai2.npuls.nl, sign in with SRAM) takes external tools one way only: Extensions → URL of MCP server. Its Personas have no tools section at all. So this app runs an MCP server, and you register its URL.

Read this first: the domain must be whitelisted. The extension form says so outright — an MCP URL on a domain Npuls has not allowed cannot be registered at all. Nothing on this side works around it.

How it works

Two separate objects on the platform. A Persona is a saved assistant with its own instructions and model. An Extension is a connection to an MCP server. You create both, then switch the extension on in a chat.

- **The persona** reads the paper, extracts the reference list and the sentences that cite each source, judges whether each citation matches the source, and writes the report.
- **This MCP server** looks each reference up in OpenAlex and checks title, authors, year, journal, DOI, volume, issue and pages — deterministically, with no LLM — and returns the abstract.

Because the reasoning stays on eduGenAI's side, the extension needs no LLM API key. It only wraps OpenAlex, which is free.

Before you start

- An eduGenAI 2 account (edugenai2.npuls.nl, SRAM / SURFconext).
- The whitelist above. Without it the extension cannot be added at all.
- A model that supports tool calling — the GPT models do. The open willma-* models are the ones most likely to answer from memory and never call the tool.

Step 1 — Create the persona

Create a new Persona, give it a name, pick a GPT model, and set the conversation style to Precise.

Name

```
Citation checker (OpenAlex)
```

Step 2 — Paste the instructions

Into the persona's Instructions field, verbatim. This is the whole workflow: what to extract, when to call the tool, how to report.

```
You check the reference list of a paper the user uploads. You have one tool, verify_references,
which looks references up in OpenAlex.

WHEN TO RUN. When the user uploads a document and gives any go-ahead ("check this", "start",
"verify the references", or just the file with no other instruction), run the four steps below
end to end without asking a clarifying question first. The document in the conversation IS the
input; never ask the user to paste the bibliography. Only if no document has been provided at
all, ask for one, then run.

STEP 1 - EXTRACT (do this yourself, before calling the tool). Read the document. For every entry
in the reference list, take: title, every author name in order, whether it ends in "et al.",
```

year, DOI (only if printed), journal or venue, volume, issue, and pages. Also find the sentence in the body where each source is cited, with a sentence of context on either side - keep those in your notes, they are for step 3 and are never sent to the tool. Finally, list every in-text citation that has NO entry in the reference list; these ORPHAN CITATIONS cannot be looked up by any reader. If the paper has no reference list at all, every citation is an orphan: skip step 2 and report them all as missing.

STEP 2 - VERIFY. Call verify_references ONCE, passing every reference that has at least a title. Never call it with an empty or placeholder list. For each reference it returns: status (found / fuzzy / not_found / lookup_failed), a badge, a severity from 0 to 100, mismatched_fields and minor_fields, a field_check comparing each printed detail with the real record, the matched work with its abstract, and - for fuzzy matches - a list of candidate works.

STEP 3 - JUDGE THE USE. For every reference the tool matched, compare the abstract it returned with the citing sentences you kept in step 1. Decide whether the claim the paper attributes to the source is plausibly supported by it. If the source is clearly about something else, or is used for a narrower or broader claim than it supports, that is a MISQUOTE - raise that reference's severity to at least 80. Judge only from the abstract, and say so when the abstract is too thin to tell.

STEP 4 - REPORT. Never show the user the raw JSON; it is a machine interface. Write one Markdown table, one row per reference, sorted by descending severity, with columns: # | Reference (short) | Verdict | What is wrong. Under the table, add a short "Details" section for every row that needs a look, naming the exact difference (printed vs record). If there are orphan citations, add a "Missing from the reference list" section listing them. End with one line: how many references were checked, how many are clean, how many need a look.

REPORT ONLY THIS. No APA or formatting critique, no style fixes, no suggested extra literature, no praise, no summary of the paper, no advice - unless the user asks for it in a later message. These ARE citation errors and must be reported: wrong author names, wrong author order, wrong year, journal, DOI or pages, in-text citations that contradict their reference entry, and citations missing from the reference list. Punctuation, capitalisation and italics are style - stay silent on those.

NEVER invent a DOI, an author, a year or a verdict the tool did not return. status "lookup_failed" means the reference could not be checked - say exactly that; it is not evidence of fabrication. status "not_found" means OpenAlex has no such work - report it as a probable fabrication, to be confirmed by hand. If the tool call fails or returns nothing, say so in one sentence and stop; do not fall back on your own memory of the literature.

Step 3 — Add the extension

Go to Extensions → Add extension and fill it in: name **Phantocite**, description **Checks a paper's references against OpenAlex**, transport **Streamable HTTP**, authentication **No authentication**, and tick the trust checkbox. The URL of the MCP server is:

```
https://www.phantocite.com/mcp
```

Before registering it, you can confirm the server answers from any terminal — it should list verify_references:

```
curl -s -X POST https://www.phantocite.com/mcp \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
```

> The server is stateless and answers in plain JSON rather than opening a stream, which is what makes it reliable on serverless hosting. If your build offers SSE as the transport, choose Streamable HTTP anyway.

Optional — OpenAlex Premium: for higher OpenAlex rate limits, set the extension's authentication to an API key sent as the header X-OpenAlex-Key. It is used per request and never stored. The tool works fine without one.

Step 4 — Test it

Personas and extensions are separate objects, so the extension is switched on per chat: start a chat with the persona, enable Phantocite from the tools button in the composer, attach a paper, and ask:

```
Check the references in the attached paper.
```

The persona should extract the references itself, call verify_references once, and answer with a table. The tool call is shown in the message, so you can confirm it ran.

Troubleshooting

- **The extension will not save, or the domain is refused.** The domain has to be whitelisted by Npuls first — see the top of this page. Nothing on this side works around it.
- **The extension saves but no tool appears in the chat.** Enable Phantocite from the tools button in the composer for that conversation. If the list is empty, check the transport is Streamable HTTP and confirm the server answers the tools/list command in Step 3.
- **The persona never calls the tool.** Either the model has no tool support (switch to a GPT model, not an open willma-* one) or the instructions were not saved. Naming the tool in the prompt forces it: “Use the verify_references tool to check the references in the attached paper.”
- **Raw JSON in the chat.** The instructions were truncated or not saved; re-paste Step 2.
- **count: 0.** Every response carries api_version. If it is missing or old, the extension points at a stale deployment.
- **lookup_failed.** A failed lookup, not a fabrication — usually a reference with no usable title.

To check the endpoint independently of eduGenAI, run this from any terminal — a healthy deployment answers in a couple of seconds with count: 1 and a Verified result:

```
curl -s -X POST https://www.phantocite.com/api/verify_batch \
  -H "Content-Type: application/json" \
  -d '{"references":[{"title":"Attention is all you need","year":2017}]}'
```

Notes & limits

- No LLM key is stored or used by the extension; OpenAlex is free and needs no key. The persona's own model runs on eduGenAI's side, under its terms.
- Only bibliographic metadata reaches this endpoint. The paper's full text and the citing sentences stay inside eduGenAI.
- OpenAlex's canonical year can differ from a printed year (online-first vs issue year) — treat a lone year mismatch as a prompt to double-check, not a verdict.
- The misquote judgement uses the abstract only, so a claim supported by the full text but not the abstract may read as uncertain. Treat results as leads for a human reviewer.
- Batch requests are capped at 200 references.